

Mathematical Foundations of PagedAttention and FlashDecoding for LLM Inference

Kernel Development Lab
Hardware-Level Performance Engineering

October 3, 2026

Abstract

This technical report details the mathematical formulation, memory layout, and parallel reduction schemes for PagedAttention [1] and FlashDecoding (Split-KV) [4] in autoregressive Large Language Model (LLM) inference. We formalize the non-contiguous virtual-to-physical memory translation of Key-Value (KV) cache tensors and derive numerically stable online softmax formulations [5, 2] for both single-pass and multi-split parallel reductions on NVIDIA GPUs.

1 Introduction and Problem Formulation

During the autoregressive decode phase of Transformer-based Large Language Models, a single token ($M = 1$) is generated per step. At step L , the query vector $\mathbf{q} \in \mathbb{R}^d$ for a given attention head must compute dot-product attention against all prior context tokens $t \in [0, L - 1]$ stored in the Key and Value caches:

$$\mathbf{k}_t, \mathbf{v}_t \in \mathbb{R}^d, \quad 0 \leq t < L \quad (1)$$

In standard implementations, KV caches are allocated contiguously in High-Bandwidth Memory (HBM):

$$\mathbf{K}, \mathbf{V} \in \mathbb{R}^{\text{Batch} \times \text{Heads} \times L_{\max} \times d} \quad (2)$$

This leads to two critical inefficiencies:

1. **Memory Fragmentation:** Up to 60% – 80% of GPU memory is stranded due to conservative pre-allocation for $L_{\max}$ and unpredictable request termination [1].
2. **Decode Underutilization:** For a single token ($M = 1$), an attention head is mapped to a single thread block. When L is large, Streaming Multiprocessors (SMs) suffer from low occupancy and DRAM latency bottlenecks [4].

2 Paged Memory Mapping

2.1 Logical to Physical Block Translation

Let the context length of sequence s be L_s . The sequence context is partitioned into logical blocks of uniform size B (typically $B = 16$ tokens).

For any token index $t \in [0, L_s - 1]$:

$$b_{\text{logical}}(t) = \left\lfloor \frac{t}{B} \right\rfloor \quad (3)$$

$$o(t) = t \pmod{B} \quad (4)$$

where $b_{\text{logical}}(t)$ is the logical block index and $o(t)$ is the intra-block slot offset.

A 2D block table $\mathcal{T} \in \mathbb{Z}^{S \times M_{\text{blocks}}}$ maps sequence s and logical block index to a physical block index $p \in [0, N_{\text{blocks}} - 1]$:

$$p = \mathcal{T}[s, b_{\text{logical}}(t)] \quad (5)$$

2.2 Physical Memory Address Resolution

Let the global physical cache pools be allocated as 4D tensors:

$$\mathbf{K}_{\text{pool}}, \mathbf{V}_{\text{pool}} \in \mathbb{R}^{N_{\text{blocks}} \times H \times B \times d} \quad (6)$$

For head index $h \in [0, H - 1]$ and token offset $o \in [0, B - 1]$, the exact element address offset in memory (assuming row-major order) is:

$$\text{Addr}(\mathbf{k}_t) = \text{Base}_K + (p \cdot H \cdot B \cdot d + h \cdot B \cdot d + o \cdot d) \times \text{sizeof}(\text{fp16}) \quad (7)$$

This enables dynamic page-level allocation without requiring contiguous physical HBM slabs across sequence lifetime.

3 Online Softmax and Single-Pass Decode

3.1 Standard Softmax Hazard

Standard scaled dot-product attention computes:

$$S_t = \frac{\mathbf{q} \cdot \mathbf{k}_t}{\sqrt{d}}, \quad P_t = \frac{\exp(S_t)}{\sum_{\tau=0}^{L-1} \exp(S_\tau)}, \quad \mathbf{O} = \sum_{t=0}^{L-1} P_t \mathbf{v}_t \quad (8)$$

Direct evaluation requires three passes over memory: (1) compute all S_t and write to DRAM, (2) compute global maximum $m = \max_t S_t$ and sum of exponentials, (3) normalize and accumulate with $\mathbf{v}_t$.

3.2 Online Recurrence Formulation

To execute in a single memory pass within fast on-chip SRAM (Shared Memory and Registers), we maintain running state $(m^{(t)}, l^{(t)}, \mathbf{o}^{(t)})$ following the online normalizer principle [5, 2, 3]:

$$m^{(t)} = \max(m^{(t-1)}, S_t) \quad (9)$$

$$\alpha_t = \exp(m^{(t-1)} - m^{(t)}) \quad (10)$$

$$l^{(t)} = \alpha_t \cdot l^{(t-1)} + \exp(S_t - m^{(t)}) \quad (11)$$

$$\mathbf{o}^{(t)} = \alpha_t \cdot \mathbf{o}^{(t-1)} + \exp(S_t - m^{(t)}) \mathbf{v}_t \quad (12)$$

with initial boundary conditions:

$$m^{(-1)} = -\infty, \quad l^{(-1)} = 0, \quad \mathbf{o}^{(-1)} = \mathbf{0} \quad (13)$$

After consuming all L_s tokens, the final attention output vector $\mathbf{O} \in \mathbb{R}^d$ is:

$$\mathbf{O} = \frac{\mathbf{o}^{(L_s-1)}}{l^{(L_s-1)}} \quad (14)$$

4 FlashDecoding (Split-KV) Parallel Reduction

When context length L exceeds 2048 tokens, serial block traversal within a single thread-block leaves the GPU compute cores starved. FlashDecoding splits the sequence dimension L across K parallel splits.

4.1 Stage 1: Partial Split Evaluation

The sequence $[0, L - 1]$ is partitioned into K intervals:

$$I_k = \left[\left\lfloor \frac{kL}{K} \right\rfloor, \left\lfloor \frac{(k+1)L}{K} \right\rfloor - 1 \right], \quad k \in [0, K - 1] \quad (15)$$

Each split k is executed by an independent CUDA thread block, producing:

- Local maximum: $m_k = \max_{t \in I_k} S_t$
- Local normalizer: $l_k = \sum_{t \in I_k} \exp(S_t - m_k)$
- Unnormalized accumulator: $\mathbf{o}_k = \sum_{t \in I_k} \exp(S_t - m_k) \mathbf{v}_t$

4.2 Stage 2: Cross-Split Log-Sum-Exp Merge

A second reduction kernel loads the K tuples $(m_k, l_k, \mathbf{o}_k)$ for each (s, h) pair.

The global maximum across all splits is:

$$m_{\text{global}} = \max_{0 \leq k < K} m_k \quad (16)$$

Rescaling factors are computed:

$$\beta_k = \exp(m_k - m_{\text{global}}) \quad (17)$$

The global denominator is:

$$l_{\text{global}} = \sum_{k=0}^{K-1} \beta_k \cdot l_k \quad (18)$$

The final output is computed by normalized convex combination:

$$\mathbf{O}_{\text{final}} = \frac{1}{l_{\text{global}}} \sum_{k=0}^{K-1} \beta_k \cdot \mathbf{o}_k \quad (19)$$

5 Hardware and Vectorization Invariants

1. **128-bit Vectorized Loads:** Key and Value elements are loaded using `float4` instructions, reading 8 FP16 values per transaction. d must be a multiple of 8.
2. **Warp Shuffles:** Reductions across threads inside a 32-thread warp execute using `__shfl_down_sync` instructions without shared memory latency.
3. **Shared Memory Limits:** Shared memory footprint per thread block is bounded by:

$$\text{SRAM} = d \times \text{sizeof}(\text{half}) + B \times \text{sizeof}(\text{float}) + \mathcal{O}(1) \leq 4 \text{ KB} \quad (20)$$

permitting high occupancy on sm_80 and sm_89 architectures [6].

References

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symp. Operating Syst. Princ. (SOSP ’23)*, 2023, pp. 611–626. doi: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).
- [2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Adv. Neural Inf. Process. Syst. (NeurIPS 2022)*, vol. 35, 2022, pp. 16344–16359.
- [3] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *Int. Conf. Learn. Represent. (ICLR 2024)*, 2024. arXiv: [2307.08691](https://arxiv.org/abs/2307.08691).
- [4] T. Dao, D. Haziza, F. Massa, and G. Sizov, “Flash-decoding for long-context inference,” *Stanford CRFM Blog*, Oct. 2023. [Online]. Available: <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>.
- [5] M. Milakov and N. Gimelshein, “Online normalizer calculation for softmax,” *arXiv preprint arXiv:1805.02867*, 2018. doi: [10.48550/arXiv.1805.02867](https://doi.org/10.48550/arXiv.1805.02867).
- [6] NVIDIA Corporation, *CUDA C++ Programming Guide (Release 12.x)*, Santa Clara, CA, USA, 2024. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [7] NVIDIA Corporation, *NVIDIA A100 Tensor Core GPU Architecture: Unprecedented Acceleration at Every Scale*, Whitepaper WP-10019-001_v01, Santa Clara, CA, USA, 2020. [Online]. Available: <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>.